

Privacy friendly revocation of credentials

07-05-2026

Serge Airapetjan

Marcelo Ribero Cerro

Yuliia (Lynn) Denysiuk

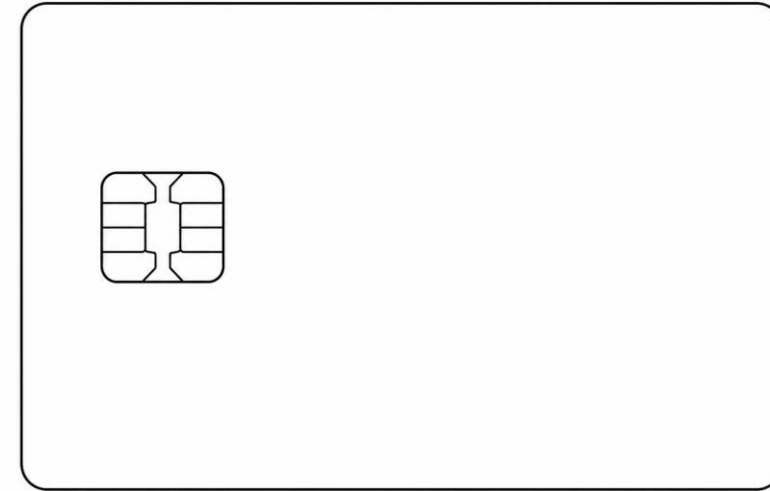
STEP-BY-STEP **AGENDA**

- Smart cards & smartphones
- Cyclic groups recap
- Zero-knowledge proofs
- Anonymous credentials & Attribute-based credentials
- Naive solutions
 - Limited lifespan
 - Blacklists with pseudonyms
 - Signature lists
- Verifier-local revocation
- Accumulators
- Discussion

SMART CARDS & SMARTPHONES

SMART CARDS & SMARTPHONES

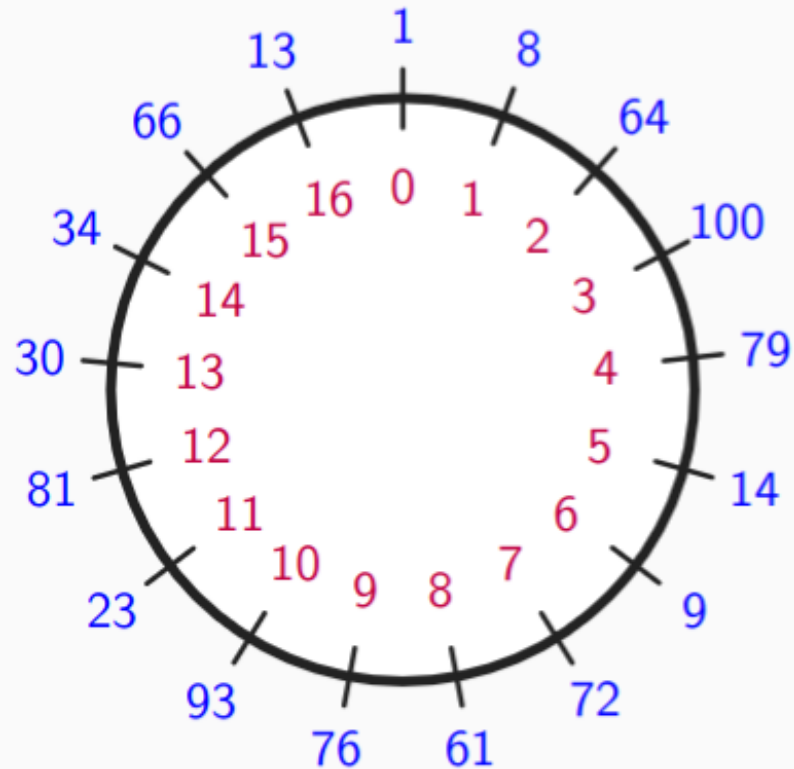
- Widely adopted: OV-chipkaart, ID cards, building access cards
- No smart cards with ABC in practice
- Limited resources
- It is possible though: Idemix and IRMA
- IRMA → YIVI



CYCLIC GROUPS RECAP

PRELIMINARIES

CYCLIC GROUPS

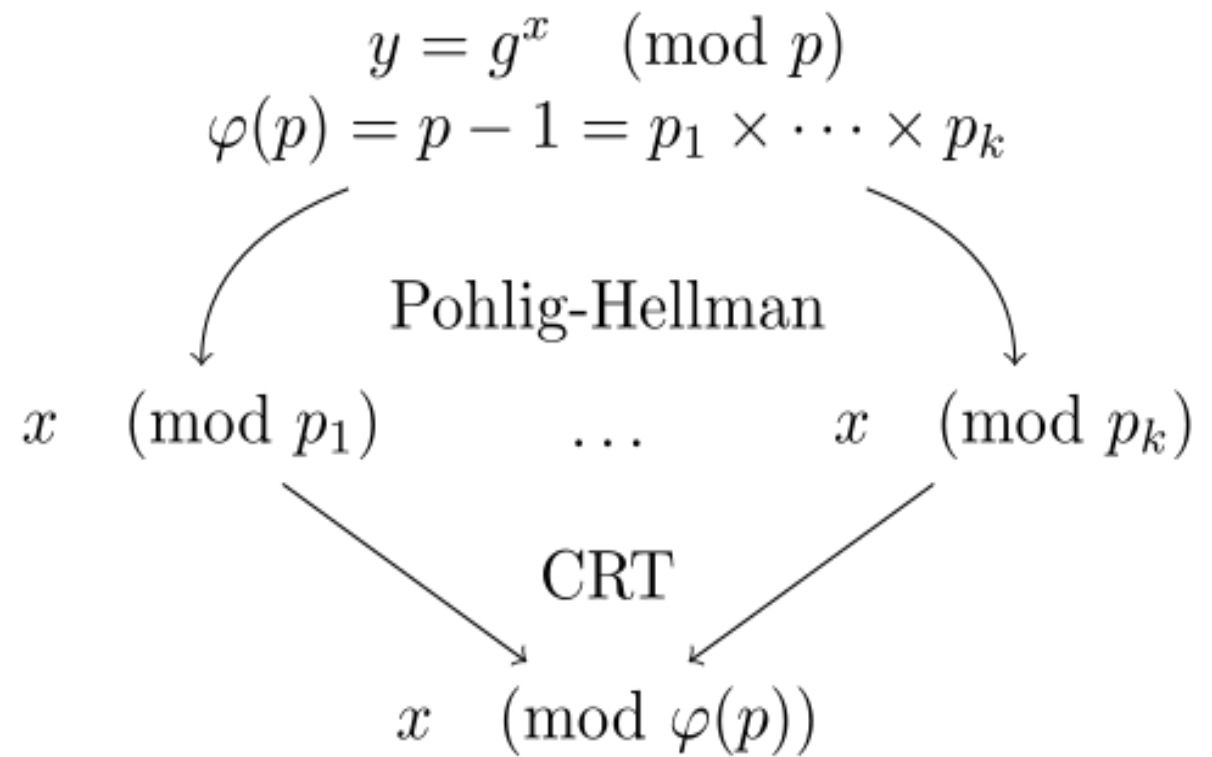


- Cyclic group G with prime order q and generator g
- Choose the **secret value** a on the red circle
- Calculate **public value** $A = g^a$

- **Example:**
- $g = 8, q = 17, G = \langle g \rangle \subseteq \mathbb{Z}_{103}^*$

PRELIMINARIES
CYCLIC GROUPS

- $A = g^a \leftarrow$ easy
- $a = \log_g A \leftarrow$ complex!



ZERO-KNOWLEDGE PROOFS

PRELIMINARIES

ZERO-KNOWLEDGE PROOFS

- Key idea: proof of knowledge without revealing it
- Cyclic groups

PRELIMINARIES

ZERO-KNOWLEDGE PROOFS

Alice

What does Alice know?
[$g, s, S = g^s$]

$r \leftarrow \text{Random}$

$R = g^r$

c

$\text{resp} = r + s \cdot c$

R

c

resp

Bob

What does Bob know?
[$g, S = g^s$]

R

$c \leftarrow \text{Random}$

resp

$$g^{\text{resp}} == R \cdot S^c$$

$$g^{r+s \cdot c} == g^r \cdot (g^s)^c$$

$$g^{r+s \cdot c} == g^r \cdot g^{s \cdot c}$$

ANONYMOUS CREDENTIALS & ATTRIBUTE-BASED CREDENTIALS

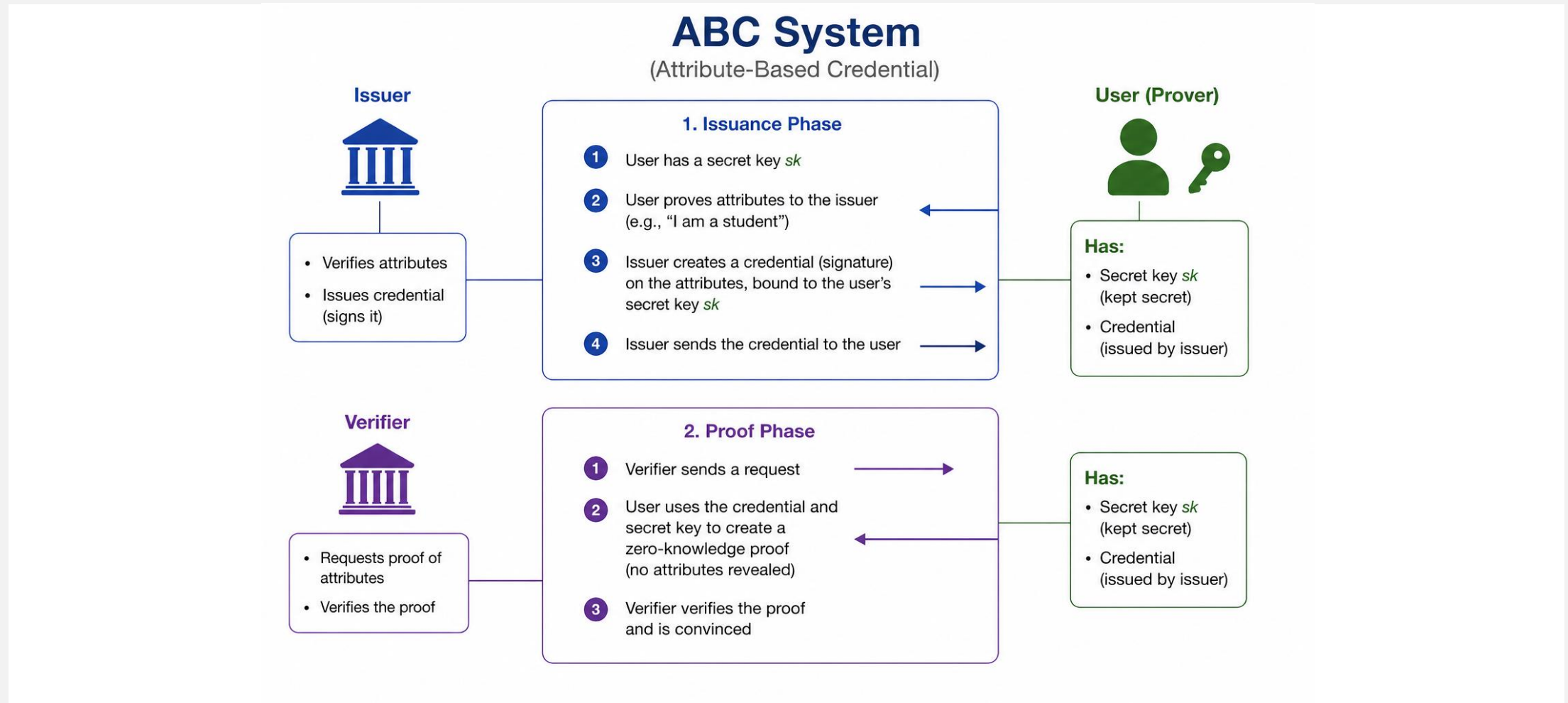
CREDENTIALS & ANONYMOUS CREDENTIALS

- Credentials
 - Qualifications
 - Documents like ID card
 - Password
- Problem: credentials reveal a lot of information
- Anonymous credentials
 - Cryptographic proof of owning a valid credential

ATTRIBUTE-BASED CREDENTIALS

- Credentials that consist of a set of attributes
 - E.g. Name, date of birth, gender, etc.
 - Signed by an "issuer"
- Choose what you reveal

INVOLVED PARTIES



SECURITY AND PRIVACY GOALS

- Unforgeability
- Unlinkability
- Selective disclosure
- Anonymity
- Non-transferability
- Revocability

REVOCATION OF CREDENTIALS

- Revocation of credentials is necessary
- Anonymous credential systems are designed to be unlinkable, so how do you revoke them?
- Each revocation algorithm comes with trade-offs

NAIVE SOLUTIONS

NAIVE SOLUTIONS 1

LIMITED LIFESPAN

- Short-term expiry date on all credentials
- Trade-off: security vs. overhead costs of generating new credentials
- Expiry date can break a security goal

NAIVE SOLUTIONS 2

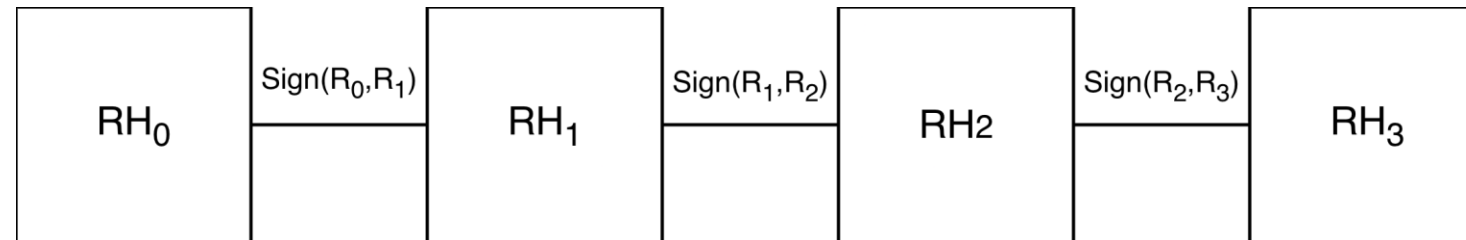
BLACKLISTS WITH PSEUDONYMS

- Pseudonym = g^k
- When credential is revoked, pseudonym is added to blacklist
- Verifier checks: Pseudonym $\in$ Blacklist
- This also breaks linkability

CAN WE DO THIS WITHOUT CAUSING LINKABILITY?

SIGNATURE LISTS

- Attach **revocation handle (rh)** to credential
- When credential is revoked, rh is added to the blacklist
- Sort blacklist: $RH_0 < RH_1 < RH_2 < RH_3$
- Each pair is signed
- Prover needs to show for own rh and a pair $(RH_i, RH_j) \in BL$:
 1. $RH_i < rh < RH_j$
 2. $\text{Sign}((RH_i, RH_j))$



PROBLEMS WITH THIS APPROACH

- Leaks information: rh lies between 2 values
- Inefficient: Elements need to be added constantly
- Signatures over pairs need to be generated

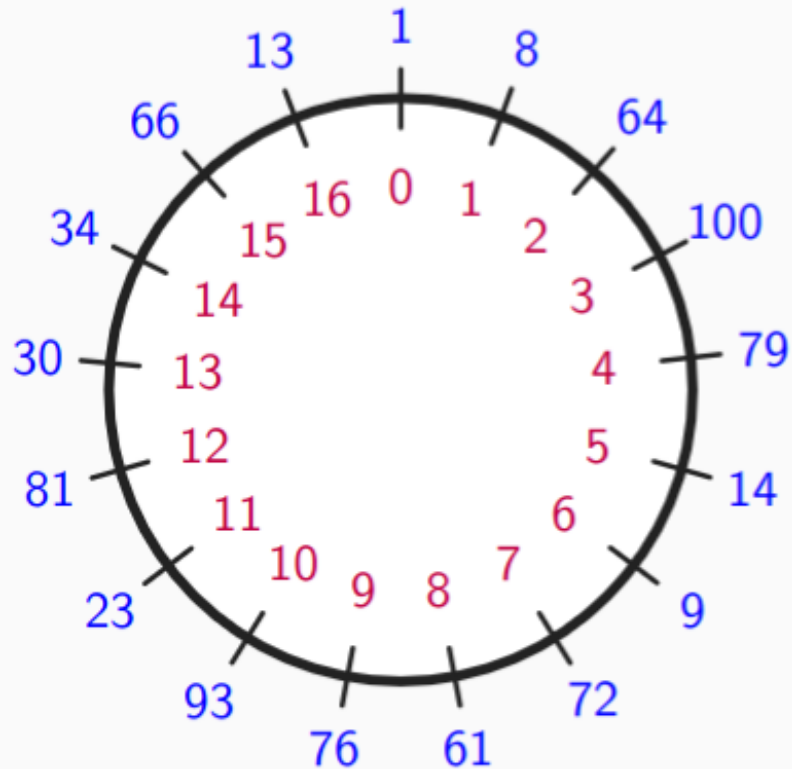
VERIFIER-LOCAL REVOCATION

RECAP

Goals

- **Basic functionality (including revocation)**
- **Privacy (unlinkability)**
- **No load on prover**
- **Control over revocation for both parties**

VERIFIER-LOCAL REVOCATION PRELIMINARIES



- Cyclic group G with prime order q
- Every credential encodes revocation value $r \in \mathbb{Z}_q^*$
- To hide r a random g is chosen
- User sends $(g, R) = (g, g^r)$
- Verifier computes g^{r_i} for every r_i in the revocation list

$$RL = \{r_1, \dots, r_k\}.$$

- Example:
- $g = 8, q = 17, G = \langle g \rangle \subseteq \mathbb{Z}_{103}^*$

Problems?

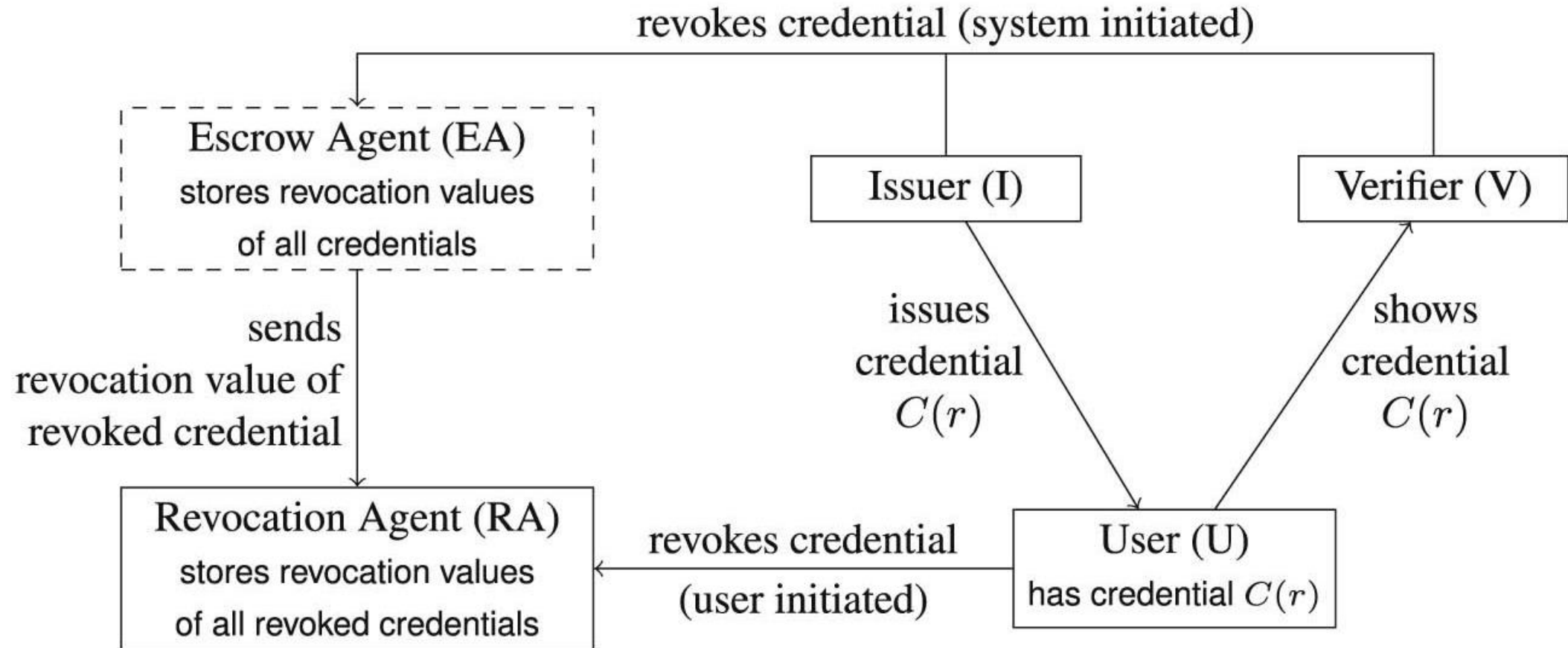
- **Complexity**
- **Traceability after revocation**
- **Per-epoch**
- **Per-verifier**
- **Semi-trusted 3rd party**

VERIFIER-LOCAL REVOCATION **PER-GROUP/PER-EPOCH**

- ε - epoch counter
- V – verifier ID
- $RL_{\varepsilon,V}$ - calculated and stored at RA (Revocation Authority)

$$RL_{\varepsilon,V} = \left\{ g_{\varepsilon,V}^{r_1}, \dots, g_{\varepsilon,V}^{r_k} \right\}.$$

VERIFIER-LOCAL REVOCATION GENERAL STRUCTURE



VERIFIER-LOCAL REVOCATION **IMPLEMENTATION**

- How user (smart card) tracks epochs?
- Estimate time!
- To get a generator for both parties we use a hash function

VERIFIER-LOCAL REVOCATION **SCHEME**

- **SetupRA()**
- **SetupEA()**
- **SetupU()**
- **SetupV()**

VERIFIER-LOCAL REVOCATION **SCHEME**

- **GetRevocationList()**
- **ObtainCredential(l)**
- **UserInitiatedRevoke(r)**
- **SystemInitiatedRevoke($id, (R, \bar{g}),$
evidence)**
- **ShowCredential(C, V)**

VERIFIER-LOCAL REVOCATION

MAIN USE CASES

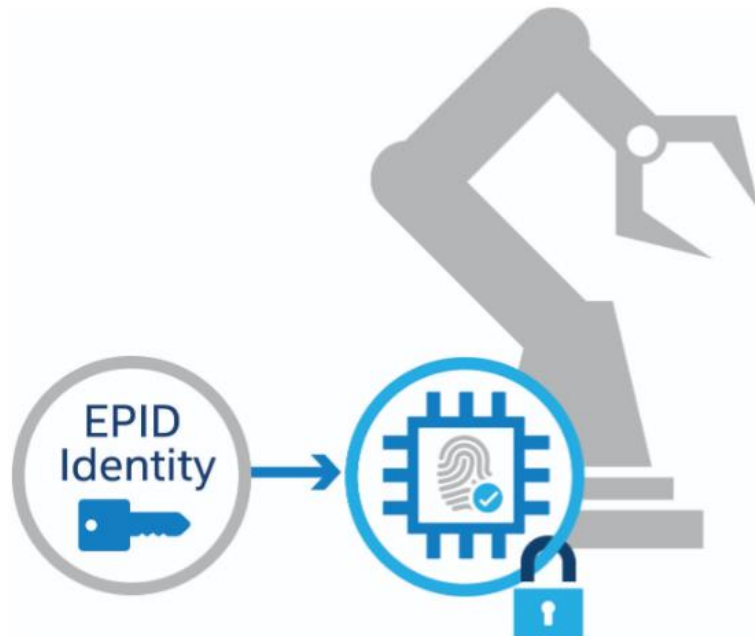
- Smart cards
- Physical access control
- Transport system



VERIFIER-LOCAL REVOCATION

MAIN USE CASES

- Already deployed!



DYNAMIC ACCUMULATORS

DYNAMIC ACCUMULATORS

WHAT ARE THEY?

Dynamic accumulator is a term used in the [permaculture](#) and [organic farming](#) literature to indicate plants that gather certain [minerals](#) or [nutrients](#) from the soil and store them in a more bioavailable form and in high concentration in their [tissues](#), then used as [fertilizer](#) or just to improve the [mulch](#).



DYNAMIC ACCUMULATORS

WHAT ARE THEY?

Dynamic accumulators are a mathematical construction that allow us to store, modify and check properties of a set into a single value. Used in Yivi/IRMA.

$$acc \leftarrow \{v_1, v_2, \dots, v_n\}$$

We will be talking about dynamic accumulators using whitelists and ECC.

Introduced by Camenisch and Lysyanskaya.

DYNAMIC ACCUMULATORS

WHAT ARE THEY?

- Initialization()
- Addition()
- Removal()
- Verification()

$$acc \leftarrow \{v_1, v_2, \dots, v_n\}$$

DYNAMIC ACCUMULATORS

INITIALIZATION

$$sk \leftarrow (\mathbb{Z}/p\mathbb{Z})$$

$$pk = sk \cdot g$$

Public $acc_o = g$

DYNAMIC ACCUMULATORS

ADDITION

$$acc_1 = (sk + v_1)acc_0$$

$$acc_2 = (sk + v_2)acc_1$$

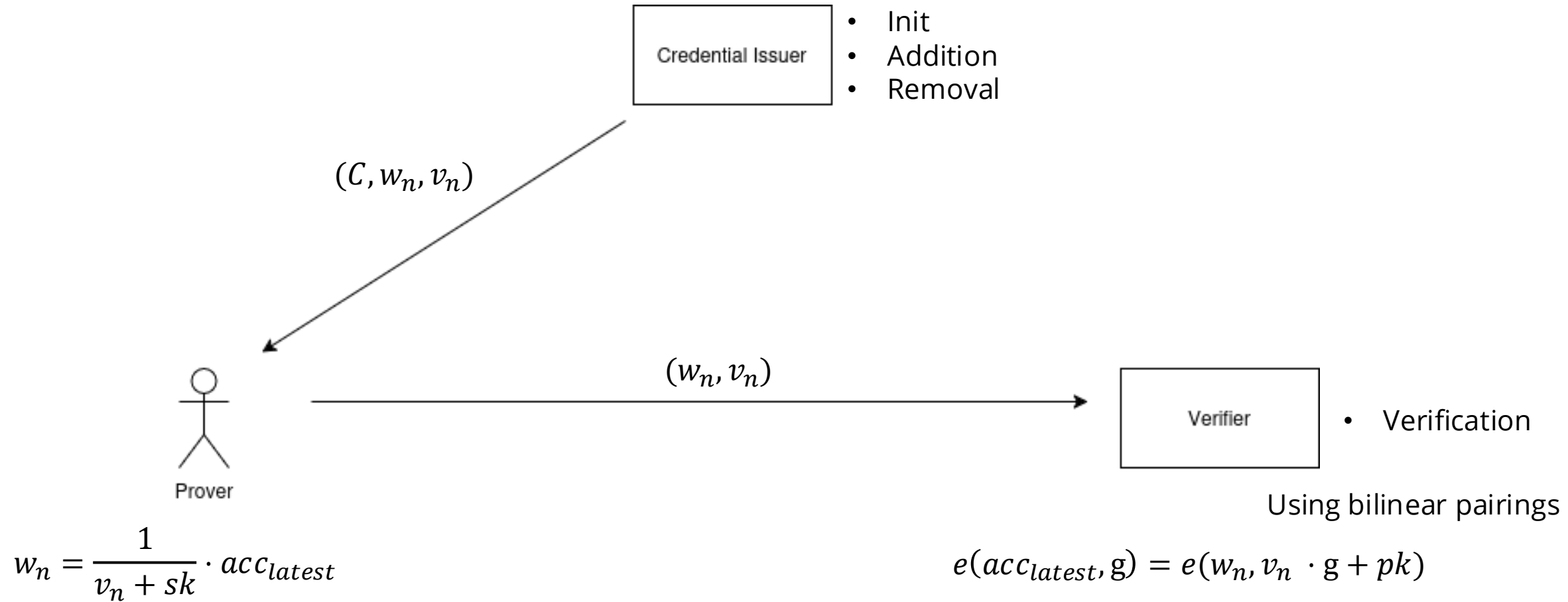
DYNAMIC ACCUMULATORS

REMOVAL

$$acc_3 = \frac{1}{sk + v_2} acc_2$$

$$acc_3 = acc_1$$

VERIFICATION



DYNAMIC ACCUMULATORS

VERIFICATION

$$e(acc_{latest}, g) = e(w_n, v_n \cdot g + pk)$$

Remember

- Secret values: sk, w_n, v_n
- $e(a \cdot x, b \cdot y) = e(a, b)^{xy}$
- $w_n = \frac{1}{v_n + sk} \cdot acc_{latest}$

But, there is a problem with this

Linkability

But, there is ANOTHER problem.
What happens if we update the accumulator?

DYNAMIC ACCUMULATORS

UPDATE ON ADDITION

$$w'_{mine} = (v_{new} - v_{mine})w_{mine} + acc_{latest}$$

DYNAMIC ACCUMULATORS
THUMBS UP



DYNAMIC ACCUMULATORS

MORE MORE MORE

- How to avoid having to update the accumulator every time an element gets added?
- Non-membership proofs – Blacklists

DISCUSSION

DISCUSSION

HOW DO THE PROTOCOLS COMPARE?

- What is used in practice? Accumulators vs. Verifier Local Revocation
- VLR was not widely adopted
- Accumulators suggest better trade-offs between privacy and efficiency/scalability
- But what about places where phones are not allowed?